

MERGE

Backend PRD — Java 21 Spring Boot Engineer

Everything you need to build the Merge EdTech backend from scratch

Version 1.0 · Backend Engineering · Confidential

Tech Stack

LAYER	TECHNOLOGY	VERSION	PURPOSE
Language	Java	21 (LTS)	Non-negotiable
Framework	Spring Boot	3.x	Modular monolith
Build	Maven	Latest	Dependency management
Database	Supabase (PostgreSQL)	15	Primary data store
Vector DB	pgvector	Latest	AI memory + similarity search
Cache	Redis	7 Alpine	XP totals, rate limits, session cache
Job Queue	BullMQ	Latest	All async processing
Code Execution	Judge0	Self-hosted	Student code execution sandbox
AI	Gemini API	Latest	Student's own token — backend only
GitHub	GitHub API + Octokit	Latest	Auto-commit approved submissions
Auth	Supabase Auth	Latest	JWT with RLS
Testing	JUnit 5 + Mockito	Latest	80% coverage minimum
Integration Tests	Testcontainers	Latest	Real database in CI
Contract Tests	Pact	Latest	Frontend/backend API contracts
Load Tests	k6	Latest	100 concurrent students
Security Tests	OWASP ZAP	Latest	Every deployment
Containerization	Docker	Latest	Full dev stack
CI/CD	GitHub Actions	Latest	Full pipeline
Error Tracking	Sentry	Latest	Production monitoring

Module Structure

Modular monolith. Modules communicate ONLY through service interfaces. Never through direct repository access across module boundaries.

Identity Module	Entities: Student, ScoutAssessment, PersonalisationProfile Responsibility: Authentication, Scout assessment, personalisation profile, token encryption
Curriculum Module	Entities: Stage, Concept, ConceptContent, LearningResource, SyntaxExercise, MergeLabsCompany Responsibility: Concept sequencing, content management, stage definitions
Assessment Module	Entities: Drill, Submission, ComprehensionCheck, Build, BuildSubmission, GraduationAssessment Responsibility: Drill management, submission processing, comprehension orchestration, Build evaluation
Progression Module	Entities: XPRecord, XPCap, StagePromotion Responsibility: XP atomic transactions, cap enforcement, two-gate promotion
Engagement Module	Entities: Session, WeeklyMomentum, Season, SeasonBadge Responsibility: Session tracking, momentum calculation, season management, disengagement detection
Feedback Module	Entities: CleanCodeFeedback, PeerReview, SeniorReview, Reflection Responsibility: Code quality feedback, peer review, reflection generation
Competency Module	Entities: Competency, CompetencyEvidence, StudentCompetency, EngineeringIdentity Responsibility: Evidence tracking, competency status, Engineering Identity assembly
AI Module	Entities: AIOrchestrationService + 9 prompt roles, AudioRecord Responsibility: All Gemini API calls, pgvector queries, all AI generation
Integration Module	Entities: GitHubService, Judge0Service, CloudflareR2Service Responsibility: GitHub commits, code execution, external service connections

API Endpoints

IDENTITY ENDPOINTS

METH OD	PATH	PURPOSE	RESPONSES
POST	/api/v1/auth/register	Student registration	200 student 400 validation 409 email exists
POST	/api/v1/auth/login	Login + JWT	200 token 401 invalid 429 rate limit
POST	/api/v1/auth/refresh	Silent token refresh	200 new token 401 expired refresh
POST	/api/v1/auth/logout	Invalidate token	200 ok
GET	/api/v1/students/me	Get own profile	200 student 401
PUT	/api/v1/students/me	Update profile	200 updated 400 401
POST	/api/v1/students/me/gemini-token	Store encrypted Gemini token	200 ok 400 invalid token
POST	/api/v1/students/me/github/connect	GitHub OAuth connect	200 ok 400 invalid code
POST	/api/v1/scout/complete	Submit Scout assessment	200 profile generated 400 incomplete

CURRICULUM ENDPOINTS

METH OD	PATH	PURPOSE	RESPONSES
GET	/api/v1/stages	All stages	200 stages[]
GET	/api/v1/stages/{stageType}	Stage details	200 stage 404
GET	/api/v1/stages/{stageType}/concepts	Concepts for stage	200 concepts[]
GET	/api/v1/concepts/{id}	Concept details + content	200 concept 404 403 not unlocked
GET	/api/v1/concepts/{id}/resources	Learning resources + books	200 resources[]
GET	/api/v1/concepts/{id}/syntax-exercise	Syntax exercise (Cadet only)	200 exercise 404 not Cadet
POST	/api/v1/syntax-exercises/{id}/submit	Submit syntax answer	200 correct/incorrect
GET	/api/v1/students/me/curriculum/progress	Full curriculum progress	200 progress

ASSESSMENT ENDPOINTS

METH OD	PATH	PURPOSE	RESPONSES
GET	/api/v1/concepts/{id}/drills	Get drills for concept	200 drills[] 403 not unlocked
POST	/api/v1/drills/{id}/submit	Submit Drill solution	200 result 400 missing fields 409 duplicate 408 timeout 403 not unlocked
GET	/api/v1/drills/{id}/hint	Request hint	200 hint 403 no hints at this stage
POST	/api/v1/comprehension-checks/{id}/submit	Submit check answers	200 passed/failed 400 timer expired
GET	/api/v1/builds/current	Get current Build	200 build 403 not unlocked yet
POST	/api/v1/builds/{id}/submit	Submit Build	200 gate results 400 missing 409 duplicate
GET	/api/v1/graduation-assessments/{stageType}	Get graduation assessment	200 assessment 403
POST	/api/v1/graduation-assessments/{stageType}/submit	Submit graduation response	200 passed/failed

PROGRESSION ENDPOINTS

METH OD	PATH	PURPOSE	RESPONSES
GET	/api/v1/students/me/xp	XP summary	200 xp totals by stage
GET	/api/v1/students/me/xp/history	XP history	200 xp_records[]
GET	/api/v1/students/me/stage/promotion-status	Check promotion eligibility	200 eligible/ineligible + missing requirements
POST	/api/v1/students/me/stage/promote	Execute promotion	200 promoted 403 requirements not met 409 already promoted

ENGAGEMENT ENDPOINTS

METH OD	PATH	PURPOSE	RESPONSES
POST	/api/v1/sessions/start	Start session with mood	200 session 400 invalid mood
POST	/api/v1/sessions/{id}/end	End session	200 summary
GET	/api/v1/students/me/momentum	Weekly momentum state	200 momentum
GET	/api/v1/cohort/momentum	Cohort momentum states	200 cohort[]
POST	/api/v1/resources/{id}/complete	Mark resource complete	200 xp awarded 200 already completed (no XP)

POST	/api/v1/audio/{id}/listened	Mark audio listened	200 ok
------	-----------------------------	---------------------	--------

FEEDBACK ENDPOINTS

METHOD	PATH	PURPOSE	RESPONSES
GET	/api/v1/submissions/{id}/feedback	Get Clean Code feedback	200 feedback 202 still generating
POST	/api/v1/peer-reviews	Submit peer review	200 ok 403 cannot review own 400 validation
GET	/api/v1/peer-reviews/pending	Get pending reviews to complete	200 submissions[]

COMPETENCY ENDPOINTS

METHOD	PATH	PURPOSE	RESPONSES
GET	/api/v1/students/me/competencies	Full competency profile	200 competencies[]
GET	/api/v1/students/me/engineering-identity	Full Engineering Identity	200 identity
GET	/api/v1/students/me/competencies/{competencyId}/evidence	Evidence for competency	200 evidence[]

Bad Paths — Must Handle

The backend must handle every one of these. No exceptions. No 500 errors for predictable bad paths.

SCENARIO	HTTP STATUS	RESPONSE BODY	HANDLING
Duplicate submission (same idempotency key)	409	{ originalSubmission }	Check key before processing. Return original.
Submission for locked concept	403	{ unlockRequirements }	Check concept sequence. Return what is needed.
Code compilation error	400	{ compilationError, stderr }	Judge0 returns COMPILATION_ERROR. Pass stderr to student.
Code timeout — 2 seconds	408	{ error: "timeout" }	Judge0 TIME_LIMIT_EXCEEDED. Return clear message.
Comprehension timer expired (server-side)	400	{ timerExpired: true }	Compare submittedAt to serverDeadline. Reject if over.
XP cap already reached	200	{ awarded: 0, capped: true }	Award remainder only. Never over cap. Never error.
Concurrent XP awards	N/A	N/A	SELECT FOR UPDATE. Retry on conflict. Atomic.
Gemini API unavailable	200	Fallback content	Query pgvector for similar cached output. Use as fallback.
GitHub rate limit	200 async	N/A	Queue retry after 60s. Max 3 retries. Log on failure.
Student not in correct stage	403	{ currentStage, requiredStage }	Stage gate check on every protected endpoint.
Peer review of own submission	403	{ error: "self-review not allowed" }	Check reviewer != submission.studentId.
Promotion without meeting gates	403	{ missingXP, missingBuildScore }	Return exactly what is missing. Never vague.
Invalid mood value	400	{ validValues: [...] }	Validate against SessionMood enum before processing.
Missing test suite on submission	400	{ error: "TDD required" }	Reject immediately. Check before sending to Judge0.

BullMQ Async Jobs

Everything that does not need to happen synchronously goes into BullMQ. David gets an immediate 200 response. Jobs process in the background.

JOB NAME	TRIGGERED BY	WHAT IT DOES	RETRY STRATEGY
GITHUB_COMMIT	Drill/Build approval	Decrypt token. Commit code to merge-portfolio repo. Store commit hash.	3 retries, 60s delay. Log on final failure.
CLEAN_CODE_FEEBACK	Comprehension check pass	AI reviews code against stage rubric. Stores CleanCodeFeedback. Extracts evidence.	2 retries. Use pgvector fallback on Gemini failure.
PERSONALISATION_UPDATE	Session end	AI analyses session patterns. Updates PersonalisationProfile embedding.	1 retry. Non-critical — skip if fails.
COMPETENCY_SIGNAL	Submission/Check/Review	Extract competency signals. Update StudentCompetency status. Update EngineeringIdentity.	3 retries. Critical — must not lose evidence.
BUILD_PRD_GENERATION	Build unlocked	AI generates personalised Build PRD from Drill history. Stores Build record.	3 retries. Block Build display until complete.
AUDIO_GENERATION	Exhausted session	AI generates audio script. Web Speech API converts. Stores AudioRecord.	2 retries. Skip if fails — mood can continue without.
MOMENTUM_CALCULATION	Every Monday 00:00	Calculate WeeklyMomentum for all students. Update states. Flag OFFLINE for reach-out.	No retry — scheduled. Log errors only.
DISENGAGEMENT_CHECK	Every 48 hours per student	Check momentum state. Generate reach-out if BLOCKED/OFFLINE. Send via Intercom.	1 retry. Non-critical.
SEASON_LOCK	Semester end date	Lock all rankings. Award SeasonBadges to top 10% and 25%. Award bonus XP.	No retry — critical manual trigger if fails.
SPOT_CHECK_PEER_REVIEW	Random % of peer reviews	Senior engineer review of peer review quality. Reverse XP if fails. Flag student.	1 retry. Human escalation on failure.

Critical Engineering Rules

These are non-negotiable. The CI pipeline will fail if violated.

Rule 1	Gemini token NEVER in frontend. Encrypted in Supabase. Decrypted ONLY in TokenEncryptionService. Backend makes all Gemini calls.
Rule 2	XP NEVER awarded from frontend. All XP calculation server-side. All XP awards in atomic transactions with SELECT FOR UPDATE.
Rule 3	Comprehension check timer enforced SERVER-SIDE. server_deadline column. Any answer after deadline returns 400. Client timer is UI only.
Rule 4	Every submission has a unique idempotency key. Duplicate requests return 409 with original submission. Check BEFORE processing.
Rule 5	Judge0 must run with ENABLE_NETWORK=false. CPU limit 2 seconds. Memory limit 256MB. No exceptions.
Rule 6	All async tasks via BullMQ. GitHub commits, AI generation, feedback, profile updates, competency signals. Never synchronous.
Rule 7	Query pgvector BEFORE every Gemini call. Never call Gemini without context retrieval. Context makes personalisation work.
Rule 8	Row Level Security on ALL student tables. Students can only read their own data. auth.uid():text = student_id::text policy on every table.
Rule 9	AES-256 encryption for all tokens. Gemini token. GitHub OAuth token. Encryption key in environment variable only. Never in code or database.
Rule 10	80% test coverage minimum. Enforced in CI. Build fails below this. Every service method has a unit test. Every bad path tested.
Rule 11	Modules communicate via service interfaces ONLY. Never direct repository access across module boundaries. Ever.
Rule 12	PWA service worker caches passive content only. Never caches API responses. Never caches submissions. Never caches auth tokens.

Docker Setup

```
version: '3.8' services: backend: build: ./backend ports: ["8080:8080"] env_file:
.env environment: - JUDGE0_URL=http://judge0:2358 - REDIS_URL=redis://redis:6379
depends_on: [redis, judge0] frontend: build: ./frontend ports: ["3000:3000"]
env_file: .env depends_on: [backend] redis: image: redis:7-alpine ports:
["6379:6379"] volumes: [redis_data:/data] judge0: image: judge0/judge0:latest
ports: ["2358:2358"] environment: - REDIS_URL=redis://judge0-redis:6379 -
DATABASE_URL=postgres://judge0:judge0@judge0-db:5432/judge0 -
ENABLE_NETWORK=false - CPU_TIME_LIMIT=2 - MEMORY_LIMIT=262144 depends_on:
[judge0-db, judge0-redis] judge0-db: image: postgres:15-alpine environment:
POSTGRES_DB: judge0 POSTGRES_USER: judge0 POSTGRES_PASSWORD: judge0 judge0-redis:
image: redis:7-alpine volumes: redis_data:
```